

REpresentational State Transfer API

Redes de Datos
Tecnicatura Universitaria en Inteligencia Artificial
Facultad de Cs Exactas Ingeniería y Agrimensura
Universidad Nacional de Rosario

Mg. Ing Juan Pablo Michelino

Índice

Índice	1
Introducción	1
¿Qué es una API?	2
Estilos de APIs	3
Arquitectura de APIs	3
RPC	4
SOAP	4
WebSocket	4
REST	5
Formatos de datos	6
XML	6
XML Espacio de Nombres	7
JSON	8
Sintaxis	8
JSON vs XML	8
YAML	9
Características	9
Comparativa entre los lenguajes	12
Análisis y serialización	13
REST API	14
Introducción	14
Solicitudes HTTP	14
Método HTTP	14
Universal Resource Identifier	14
Encabezados:	15
Cuerpo	15
Respuestas HTTP	15
Estado HTTP	15
Encabezado	16
Cuerpo	17
Diagrama de secuencias	17
WebHook	18
Autenticación de REST API	18
Mecanismos de autenticación	18
Mecanismos de autorización	18
Límite de velocidad de la REST API	19
Algoritmos de límite de velocidad	19
Contador dinámico:	19
Depósito de token	20
Contador de ventana fija	20
Contador de ventana deslizable	21
Fuentes	22

Introducción

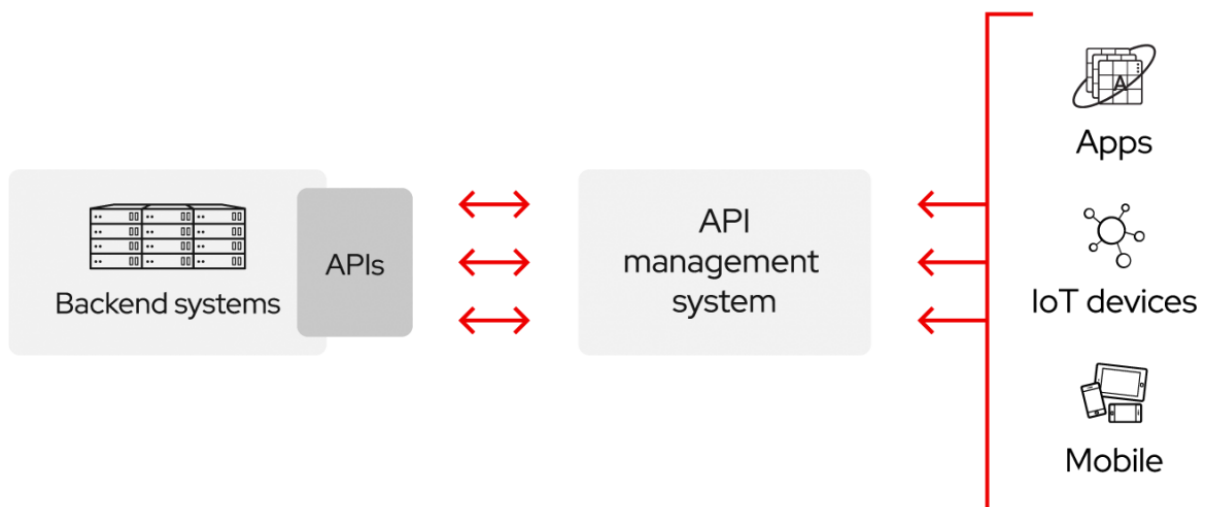
¿Qué es una API?

Una **Application Programming Interface** (o API por sus siglas) es un conjunto de definiciones y protocolos que se usa para diseñar e integrar aplicaciones. Permiten que aplicaciones se comuniquen con otras sin necesidad de saber cómo están implementados, simplificando así el desarrollo de las aplicaciones y ahorrando tiempo y dinero.

Las API le otorgan flexibilidad; simplifican el diseño, la administración y el uso de las aplicaciones; y ofrecen oportunidades de innovación, lo cual es ideal al momento de diseñar herramientas y productos nuevos (o de gestionar los actuales).

A veces, las API se consideran como contratos, con documentación que representa un acuerdo entre las partes: Si una de las partes envía una solicitud remota con cierta estructura en particular, esa misma estructura determinará cómo responderá el software de la otra parte.

Las API son un medio simplificado para conectar su propia infraestructura a través del desarrollo de aplicaciones nativas de la nube, pero también le permiten compartir sus datos con clientes y otros usuarios externos. Las API públicas aportan un valor comercial único porque simplifican y amplían sus conexiones con los partners y, además, pueden rentabilizar sus datos (un ejemplo conocido es la API de Google Maps).



Por ejemplo, piense en una empresa distribuidora de libros. Podría ofrecer a los clientes una aplicación de la nube que les permitiera a los empleados de la librería verificar la disponibilidad de los libros con el distribuidor. El desarrollo de la aplicación podría ser costoso, verse limitado por la plataforma, llevar mucho tiempo y requerir mantenimiento permanente.

Otra opción es que la distribuidora de libros proporcione una API para verificar la disponibilidad en el inventario. Existen varios beneficios de este enfoque:

- Permite que los clientes accedan a los datos con una API que les ayude a añadir información sobre su inventario en un solo lugar.
- La distribuidora de libros podría realizar cambios en sus sistemas internos sin afectar a los clientes, siempre y cuando el comportamiento de la API fuera el mismo.

- Con una API disponible de forma pública, los desarrolladores que trabajan para la distribuidora de libros, los vendedores o los terceros podrían desarrollar una aplicación para ayudar a los clientes a encontrar los libros que necesiten.

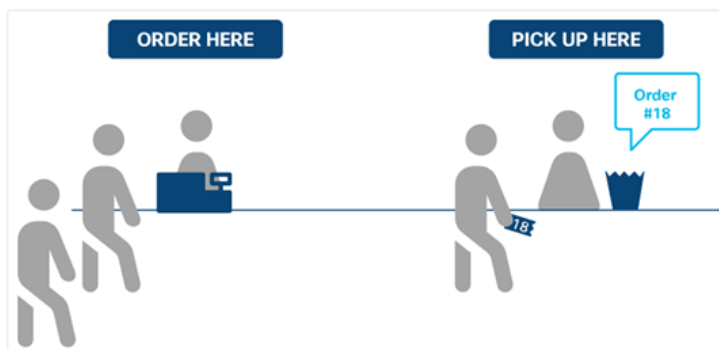
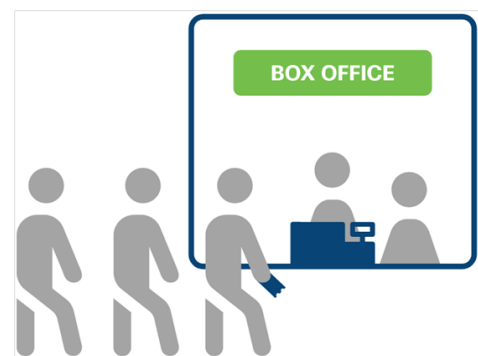
En resumen, las API le permiten habilitar el acceso a sus recursos y, al mismo tiempo, mantener la seguridad y el control. Usted decide cómo habilitar el acceso y a quiénes se lo otorga. La seguridad de las API depende de su buena gestión, lo cual incluye el uso de un gateway (puerta de enlace) de API.

Estilos de APIs

Las APIs pueden ser de dos tipos: Sincrónicas y Asincrónicas. La aplicación que las consume administra la respuesta de manera diferente dependiendo del diseño de la API.

En las **APIs sincrónicas**, los datos de la solicitud están disponibles fácilmente y responden directamente a una solicitud. Si la API está diseñada correctamente, el rendimiento de la aplicación será mejor.

Desde el lado cliente, la aplicación que realiza la solicitud de API debe esperar la respuesta antes de realizar tareas adicionales de ejecución de código.



Por otro lado, las **APIs asincrónicas** proporcionan una respuesta (sin datos) para indicar al cliente que se ha recibido la solicitud y será respondida ni bien el servidor haya procesado los datos.

Del lado cliente, las APIs asincrónicas permiten que la aplicación continúe su ejecución

sin ser bloqueada hasta que el servidor procese la solicitud, lo que resulta en un mejor rendimiento.

Con el procesamiento asincrónico, el diseño de la API en el lado del servidor define el requisito en el lado del cliente.

Arquitectura de APIs

La arquitectura de las API suele explicarse en términos de cliente y servidor: La aplicación que envía la solicitud se llama cliente y la que envía la respuesta se llama servidor.

Tomando como ejemplo un sistema que consulta pronóstico meteorológico: la base de datos meteorológica del *Servicio Meteorológico Nacional* es el servidor y la aplicación móvil es el cliente.

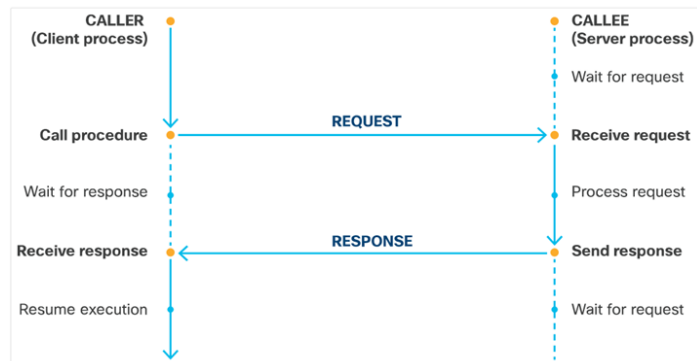
Según el momento y el motivo de su creación, las APIs pueden funcionar de cuatro maneras diferentes: RPC, SOAP, WebSocket o REST

RPC

Remote Procedure Call (o RPC por sus siglas) es un modelo de petición-respuesta que permite a una aplicación realizar una llamada a procedimiento a otra aplicación. Cuando un cliente hace una RPC,, el método se ejecuta y los resultados se devuelven.

RPC es un estilo API que se puede aplicar a diferentes protocolos de transporte como:

- XML-RPC
- JSON-RPC
- Network File System (NFS)
- Protocolo SOAP (Simple Object Access Protocol)



SOAP

Simple Object Access Protocol (o SOAP por sus siglas) es un protocolo de mensajería basado en formato de datos XML. Se utiliza para la comunicación de aplicaciones construidas en diferentes lenguajes de programación o corriendo en diferentes plataformas. El protocolo SOAP tiene tres características principales:

- Extensibilidad: Seguridad y WS-routing son extensiones aplicadas en el desarrollo.
- Neutralidad: Bajo protocolo de transporte TCP puede ser utilizado sobre cualquier protocolo de aplicación como HTTP, SMTP o JMS.
- Independencia: Permite cualquier modelo de programación.

Se conforma de tres partes:

- Envelope (Sobre), el cual define qué hay en el mensaje y cómo procesarlo.
- Conjunto de reglas de codificación para expresar instancias de tipos de datos.
- La convención para representar llamadas a procedimientos y respuestas.

WebSocket

La API de *WebSocket* es otro desarrollo moderno de la API que utiliza objetos JSON para transmitir datos.

La WebSocket admite la comunicación bidireccional entre las aplicaciones cliente y el servidor. El servidor puede enviar mensajes de devolución de llamadas a los clientes conectados, por lo que es más eficiente que la API de REST.

REST

REpresentational State Transfer (o REST por sus siglas) es un estilo de arquitectura de software para realizar una comunicación cliente-servidor.

Se apoya en el protocolo HTTP para la comunicación con el servidor y los mensajes que se envían y reciben entre ambos participantes pueden estar en XML o JSON.

En su desarrollo se establecieron seis restricciones que pueden aplicar a cualquier protocolo:

1. **Interfaz uniforme:** La interfaz entre el cliente y servidor se adhiere a cuatro principios:
 - a. Identificación de recursos
 - b. Manipulación de recursos a través de representaciones
 - c. Mensajes autodescriptivos
 - d. Hypermedia como motor del estado de la aplicación.
2. **Stateless:** REST no tiene estados, esto quiere decir que una llamada a la API es independiente de otra llamada, sin embargo se puede usar caché para reducir el tiempo de espera en consultas.
3. **Modelo de caché:** Las respuestas del servidor deben indicar si ésta puede almacenar o no en caché. En caso positivo, el cliente puede utilizar los datos de la respuesta para solicitudes posteriores.
4. **Código bajo demanda (opcional):** La información devuelta por un servicio REST puede incluir código ejecutable (por ejemplo, javascript) o enlaces destinados a extender de manera útil la funcionalidad del cliente.
5. **Sistema de capas:** Consta de diferentes capas jerárquicas en las que cada una proporciona servicios sólo a la capa superior.
6. **Cliente-Servidor:** El servidor hace el procesamiento del API y expone los recursos a uno o varios clientes, que pueden ser una aplicación de escritorio, una página web, etc. El cliente debe ser independiente del servidor y toda comunicación a él se debe dar mediante el API.

Formatos de datos

XML, JSON y YAML son los lenguajes de serialización de datos más populares.

La serialización de datos consiste en un proceso de codificación de un objeto en un medio de almacenamiento (como puede ser un archivo, o un buffer de memoria) con el fin de transmitirlo a través de una conexión en red como una serie de bytes.

XML

eXtensible Markup Language (o XML por sus siglas) traducido como “Lenguaje de Marcado Extensible”, deriva de *Structured Generalized Markup Language* (SGML) y también es el padre de *HyperText Markup Language* (HTML). Es un metalenguaje desarrollado por W3C que permite definir *etiquetas* para almacenar datos en forma legible. Permite definir la gramática de lenguajes específicos para estructurar documentos grandes.

Los nombres de los archivos suelen tener extensión “.xml” y, a diferencia de otros lenguajes, XML da soporte a bases de datos; siendo útil cuando varias aplicaciones deben comunicarse entre sí o integrar información.

Un archivo XML tiene una apariencia como la siguiente:

```
<?xml version="1.0" encoding="UTF-8"?>
<!-- Instance list -->

<Configurations>

  <Config>
    <Name>Ingress</Name>
    <Value>data/input</Value>
  </Config>

  <Config>
    <Name>Egress</Name>
    <Value>data/output</Value>
  </Config>

</Configurations>
```

En el ejemplo anterior, excepto las dos primeras líneas de un documento, el resto se considera como el **cuerpo del documento**. El **prólogo XML** es la primera línea del archivo y éstos pueden incluir comentarios utilizando la misma convención de utilizada en los documentos HTML: `<!-- comentario -->`

Los nombres de las **etiquetas** son definidos por el usuario. Éstas comprenden es el texto que encerrado entre signos “<” y “>” y, en la mayoría de los casos, tiene una etiqueta de apertura (< *etiqueta* >) y una de cierre (indicada con el mismo nombre, pero iniciando con “</”: </ *etiqueta* >).

Dentro de las etiquetas, XML permite incrustar **atributos** para transmitir información adicional.

XML no ha nacido únicamente para su aplicación en Internet, sino que se propone como un estándar para el intercambio de información estructurada entre diferentes plataformas. Se

puede usar en bases de datos, editores de texto, hojas de cálculo y casi cualquier cosa imaginable.

XML Espacio de Nombres

Los *espacios de nombres* son definidos por el IETF y otras autoridades de Internet, organizaciones y otras entidades, y sus esquemas se alojan normalmente como documentos públicos en la Web. Éstos se declaran usando el atributo XML reservado **xmlns** y se identifican mediante nombres uniformes de recursos (Uniform Resource Names-URI) para hacer que los documentos persistentes sean accesibles sin que el buscador tenga que preocuparse por su ubicación.

```
<rpc message-id="101" xmlns="urn:ietf:params:xml:ns:netconf:base:1.0">
  <kill-session>
    <session-id> 4 </session-id>
  </kill-session>
</rpc>
```

El ejemplo anterior muestra el uso de un espacio de nombres para afirmar que el contenido de una llamada a procedimiento remoto XML debe interpretarse de acuerdo con el estándar NETCONF 1.0 heredado.

Otro ejemplo de espacio de nombres sería una instancia XML que contuviera referencias a un cliente y a un producto solicitado por este. Tanto el elemento que representa el cliente como el que representa el producto pueden tener un elemento hijo llamado "*numero_ID*". Las referencias al elemento "*numero_ID*" podrían ser ambiguas, salvo que los elementos, con igual nombre pero significado distinto, se llevaran a espacios de nombres distintos que los diferencien.

```
<?xml version="1.0"?>
<cli:cliente
xmlns:cli='http://es.wikipedia.org/wiki/Espacio_de_nombres_XML/cliente'
xmlns:ped='http://es.wikipedia.org/wiki/Espacio_de_nombres_XML/pedido'>
  <cli:numero_ID>1232654</cli:numero_ID>
  <cli:nombre>Fulanito de Tal</cli:nombre>
  <cli:telefono>99999999</cli:telefono>
  <ped:pedido>
    <ped:numero_ID>6523213</ped:numero_ID>
    <ped:articulo>Caja de herramientas</ped:articulo>
    <ped:precio>187,91</ped:precio>
  </ped:pedido>
</cli:cliente>
```

El alcance de la declaración de un prefijo de espacio de nombres comprende desde la etiqueta de inicio de un elemento XML, en la que se declara, hasta la etiqueta final de dicho elemento XML. En las etiquetas vacías, correspondientes a elementos sin "hijos", el alcance es la propia etiqueta.

JSON

JavaScript Object Notation (JSON por sus siglas) es un formato de texto sencillo para el intercambio de datos.

Se trata de un subconjunto de la notación literal de objetos de JavaScript aunque, debido a su amplia adopción como alternativa a XML, se considera un formato independiente del lenguaje.

Una de las supuestas ventajas de JSON sobre XML como formato de intercambio de datos es que resulta mucho más sencillo escribir un analizador sintáctico (parser) para él.

Sintaxis

Los nombres de los archivos JSON suelen tener la extensión “.json” y su apariencia es como la siguiente:

```
{
  "edit-config":
  {
    "default-operation": "merge",
    "test-operation": "set",
    "some-integers": [2,3,5,7,9],
    "a-boolean": true,
    "more-numbers": [2.253E+2, -1.0735],
  }
}
```

Allí se puede ver una serie de **objetos** que son colecciones no ordenadas de pares de la forma **<nombre>: <valor>**, separados por comas y puestas entre llaves. El nombre tiene que ser una cadena entre comillas dobles y el valor puede ser de cualquier tipo (números, cadenas, booleanos o nulos).

Las **cadenas** representan secuencias de cero o más caracteres. Se ponen entre doble comilla y se permiten cadenas de escape.

Los **arreglos** representan una lista ordenada de cero o más valores de cualquier tipo separados por comas y encerrados entre corchetes.

A diferencia de XML y YAML, JSON no admite ningún tipo de método estándar para incluir **comentarios** en el código.

Los espacios en blanco en JSON no son significativos: se puede (o no) emplear sangría utilizando tabs o espacios como se prefiera.

JSON vs XML

A modo de ejemp, lo para comparar la sintaxis de ambos lenguajes, a continuación se muestra un ejemplo simple de definición de barra de menús usando JSON y XML

JSON:

```
{
  "menu": {
    "id": "file",
    "value": "File",
    "popup": {
      "menuitems": [
        {
          "value": "New", "onclick": "CreateNewDoc()"
        }, {
          "value": "Open", "onclick": "OpenDoc()"
        }, {
          "value": "Close", "onclick": "CloseDoc()"
        }
      ]
    }
  }
}
```

XML:

```
<menu id="file" value="File">
  <popup>
    <menuitem value="New" onclick="CreateNewDoc()" />
    <menuitem value="Open" onclick="OpenDoc()" />
    <menuitem value="Close" onclick="CloseDoc()" />
  </popup>
</menu>
```

YAML

YAML Ain't Markup Language (YAML por sus siglas) es un formato de serialización de datos, superconjunto de JSON, diseñado para una fácil legibilidad humana, inspirado en lenguajes como XML, C, Python, Perl, así como en el formato de los correos electrónicos (RFC 2822).

Como superconjunto de JSON, los analizadores YAML generalmente pueden analizar documentos JSON (pero no viceversa). Por lo tanto, YAML es mejor que JSON en ciertas tareas, incluida la capacidad de incrustar JSON directamente (incluidas las comillas) en archivos YAML.

Características

YAML fue creado bajo la creencia de que todos los datos pueden ser representados adecuadamente como combinaciones de listas, hashes (mapeos) y datos escalares (valores simples).

La sintaxis es relativamente sencilla y fue diseñada teniendo en cuenta su fácil legibilidad pero que a la vez fuese simple mapear a los tipos de datos más comunes en la mayoría de los lenguajes de alto nivel. Además, YAML utiliza una notación basada en la sangría y un conjunto de caracteres Sigil distintos de los que se usan en XML, haciendo que sea fácil componer ambos lenguajes.

```
---
edit-config:
  a-boolean: true
  default-operation: merge
  more-numbrers:
    - 225.0
    - -1.0735
  some-integers:
    - 2
    - 3
    - 4
    - 5
    - 7
    - 9
  test-poeration: set
...
```

Los archivos YAML se **abren** convencionalmente con tres guiones (--- solos en una línea) y se **cierran** con tres puntos (... igualmente) y su jerarquía se indica mediante **sangría**. Los tipos de **datos básicos** se equiparan a claves e incluyen números, cadenas, booleanos o nulos.

YAML representa fácilmente tipos de **datos más complejos**, como **mapas** que contienen varios pares clave/valor y **listas** ordenadas. Los mapas generalmente se expresan en varias líneas, comenzando con una clave de etiqueta y dos puntos, seguidos de miembros, sangrados en las líneas siguientes.

```
mymap:
  myfirstkey: 5
  mysecondkey: The quick brown fox
```

Las **listas** (matrices-arrays) se representan con miembros opcionalmente sangrados precedidos por un solo guión y espacio:

```
mylist:
  - 1
  - 2
  - 3
```

Los **mapas** y las **listas** también se pueden representar en una llamada "flow syntax", que se parece mucho a JavaScript o Python:

```
mymap: { myfirskey: 5, mysecondkey: The Quick brown fox}
mylist: [1, 2, 3]
```

Las **long strings** se representan usando una sintaxis "plegable", donde se presume que los saltos de línea son reemplazados, cuando el archivo se analiza o consume, por espacios o en una sintaxis no plegada.

```
mylongstring: >
  This is my long string
  which will end un with no linebreaks in it
myotherlongstring: |
  This es my other long string
  wich will en up with linebreaks as in the original
```

Los **comentarios** en YAML se pueden insertar en cualquier lugar excepto en un literal de cadena larga, y están precedidos por el signo numeral (#) y un espacio.

```
# This is a comment
```

YAML tiene muchas más características, la mayoría de las veces encontradas al usarlo en el contexto de lenguajes específicos como Python, o al convertir a JSON u otros formatos. YAML 1.2 admite esquemas y etiquetas, que se pueden utilizar para desambiguar la interpretación de valores. Por ejemplo, para forzar que un número se interprete como una cadena, puede usar `!!str <string>`, que es parte del esquema "Failsafe" de YAML

```
mynumericstring: !!str 0.1415
```

Comparativa entre los lenguajes

	XML	JSON	YAML
Formato de datos.	Es un lenguaje de marcas que usa etiquetas para definir los elementos y almacena datos en una estructura de árbol.	Formato de datos que los almacena como un mapa con pares clave/valor.	Formato de datos que permite la representación de datos tanto en formato de lista o secuencia como en forma de mapa con pares clave/valor.
Tabulaciones	No considera las tabulaciones.	Usa tabulaciones para la sangría.	YAML usa un guión (-) seguido de un espacio para la sangría.
Comentarios	Acepta el uso de comentarios dentro del documento	No admite comentarios.	Acepta comentarios dentro del documento.
Tipos de datos	Admite tipos de datos complejos, como gráficos, imágenes y otros tipos de datos no primitivos.	Solo admite cadenas, números, matrices, booleanos y objetos.	Admite tipos de datos complejos, como marcas de fecha y hora, secuencias, valores anidados y recursivos, y tipos de datos primitivos.
Legibilidad	Difícil de leer e interpretar.	Fácil de leer e interpretar	Mucho más fácil de leer e interpretar.
Usabilidad y propósito	Se utiliza para el intercambio de datos	Es mejor como formato de serialización y se utiliza para proporcionar datos a las APIs.	Más adecuado para configuraciones.
Velocidad	Es voluminoso y lento en el análisis, lo que lleva a una transmisión de datos relativamente lenta.	Considerablemente más pequeños que los archivos XML El motor JavaScript lo analiza rápidamente lo que permite una transmisión de datos más rápida.	Como superconjunto de JSON, también ofrece una transmisión de datos más rápida, pero se usa en un escenario diferente al de JSON.

Análisis y serialización

Examinar significa analizar un mensaje, dividiéndolo en sus componentes y comprendiendo sus propósitos en contexto. La serialización es más o menos lo opuesto al análisis.

Los lenguajes de programación populares como Python generalmente incorporan funciones de análisis fáciles de usar que pueden aceptar datos devueltos por una función de E/S y producir una estructura de datos interna semánticamente equivalente que contenga datos escritos válidos.

En el lado saliente, contienen serializadores que convierten las estructuras de datos internas en mensajes semánticamente equivalentes formateados como cadenas de caracteres.

REST API

Introducción

Una REST API se comunica a través de HTTP utilizando los mismos conceptos de ese protocolo:

- Solicitudes y respuestas HTTP
- Verbos HTTP
- Código de estado HTTP
- Encabezados y cuerpo HTTP



A continuación veremos cada uno de esos conceptos:

Solicitudes HTTP

Las solicitudes de API REST son *request HTTP* con la que una aplicación (cliente) pide al servidor que realice una función. Se componen de cuatro partes principales:

- Método HTTP.
- Universal Resource Identifier (URI).
- Encabezado.
- Cuerpo.

Método HTTP

Los **Métodos HTTP** son los estándares de comunicación para los servicios web a los que se solicita la acción para el recurso dado. La asignación sugerida del método es la siguiente:

- POST: Crea un objeto o recurso nuevo.
- GET: Recupera detalles del recurso del sistema.
- PUT: Reemplaza o actualiza un recurso existente.
- PARCHE: Actualiza algunos detalles de un recurso existente
- DELETE: Remueve un recurso del sistema.

Universal Resource Identifier

El **Universal Resource Identifier** (URI por sus siglas), también conocido como **Universal Resource Locator** (URL), identifica qué recurso desea manipular el cliente. Está compuesto de la siguiente forma:

http://localhost:8080/v1/books/?q=Query

La primera parte, el **esquema**, especifica qué protocolo se debe usar (http o https). Le sigue la **autoridad**, que consta de dos partes: **host** (en el ejemplo *localhost*) y **puerto** (en el ejemplo *8080*). Luego viene la **Ruta de Acceso**, que representa la ubicación del recurso,

los datos u objeto que se va a manipular en el servidor. Finalmente está la **Consulta**, que proporciona detalles adicionales sobre el ámbito, el filtrado o una solicitud.

Encabezados:

Los encabezados tienen el formato de pares nombre-valor separados por dos puntos (:), *[nombre]: [valor]*. Hay dos tipos de encabezados:

1. Encabezados de solicitud: Incluye información adicional que no esté relacionada con el contenido del mensaje.

Clave	Descripción	Valor de ejemplo
Autorización	Proporciona credenciales para autorizar la solicitud	DMFNCFUDDP2YWDYYW básico

2. Encabezados de entidad: Información adicional que describe el contenido del cuerpo del mensaje.

Clave	Descripción	Valor de ejemplo
Tipo de contenido	Especifica el formato de los datos en el cuerpo.	Application/JSON

Cuerpo

El cuerpo de la solicitud es opcional dependiendo del método y contiene los datos correspondientes al recurso que el cliente desee manipular. Las solicitudes que utilizan los métodos POST, PUT y PATCH suelen incluir un cuerpo.

Si los datos se proporcionan en el cuerpo, entonces el tipo de datos debe especificarse en el encabezado mediante la clave de Content-Type.

Respuestas HTTP

Las respuestas de la REST API son respuestas HTTP que comunican los resultados de la solicitud de un cliente. Se componen de tres componentes principales:

- Estado HTTP
- Encabezado
- Cuerpo

Estado HTTP

El código de estado HTTP ayuda al cliente a determinar si la solicitud fue recibida, si fue rechazada, el motivo del error y a veces puede proporcionar sugerencias para solucionar el problema.

Ellos constan de tres dígitos, donde el primero es la categoría de respuesta y los otros dos son asignados en orden numérico.

Hay cinco categorías diferentes de códigos de estado HTTP:

- **1xx** (Informativo): Con fines informativos, las respuestas no contienen un cuerpo
- **2xx** (Éxito): El servidor recibió y ha aceptado la solicitud. Ejemplo de respuestas son:

- 200 (Aceptada): La solicitud se realizó correctamente y normalmente contiene una carga útil (cuerpo)
- 201 (Creada): Se cumplió la solicitud y se creó el recurso que fue solicitado
- 202 (Aceptada): La solicitud ha sido aceptada para su procesamiento y está en proceso.
- **3xx** (Redirección): El cliente tiene que tomar una acción adicional para completar la solicitud.
- **4xx** (Error de cliente) La solicitud contiene un error como sintaxis incorrecta o entrada no válida. Las respuestas 4xx más comunes son:
 - 400 (No válida): La solicitud no se procesa debido a un error con la solicitud
 - 401 (No autorizada): La solicitud no tiene credenciales de autenticación válidas para realizar la solicitud
 - 403 (Prohibida): La solicitud ha sido entendida pero ha sido rechazada por el servidor
 - 404 (No encontrada): No se puede cumplir la solicitud porque la ruta de acceso del recurso de la solicitud no se encontró en el servidor.
- **5xx** (Error del servidor): No se pueden cumplir las solicitudes válidas. Algunos ejemplos de respuestas 5xx son:
 - 500 (Error del servidor interno): No se puede cumplir la solicitud debido a un error del servidor.
 - 503 (Servidor no disponible): No se puede cumplir la solicitud porque actualmente el servidor no puede manejar la solicitud.

Encabezado

El encabezado de la respuesta proporciona información adicional, entre el servidor y el cliente, en el formato de par nombre-valor separado por dos puntos (:), **[nombre]:[valor]**. Hay dos tipos de encabezados:

- **Encabezados de respuesta** que contienen información adicional que no está relacionada con el contenido del mensaje.
Los encabezados de respuesta típicos para una solicitud de API REST incluyen:

Clave	Descripción	Valor ejemplo
Set-Cookie	Se utiliza para enviar Cookies desde el servidor	JSESSIONID=30A9DN810FQ428P; Ruta=/
Control de Cache	Especificar directivas que DEBEN ser obedecidas por todos los mecanismos de almacenamiento en el caché	Control de caché: max-edad=3600, público

- **Encabezados de entidad** que contiene información adicional que describe el contenido del cuerpo del mensaje.
Un encabezado de entidad común especifica el tipo de datos que son devueltos:

Clave	Descripción	Valor ejemplo
Tipo de contenido	Especificar el formato de los datos en el cuerpo	Application/JSON

Cuerpo

El cuerpo de la respuesta depende del método de solicitud y contiene los datos correspondientes al recurso que el cliente desee manipular. Si los datos se proporcionan en el cuerpo, entonces el tipo de datos debe especificarse en el encabezado mediante la clave de Content-Type.

El cuerpo de respuesta puede estar paginado o comprimido:

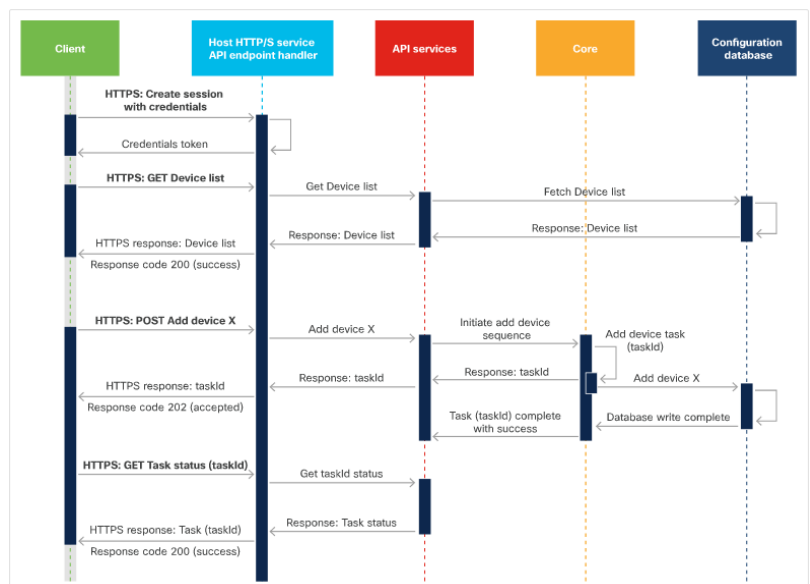
- La **paginación de respuestas** permite dividir los datos en fragmentos. La mayoría de las API que implementan la paginación utilizan el parámetro de consulta para especificar qué página devolver en la respuesta.
- Los datos **comprimidos** reducen la gran cantidad de datos que no se pueden paginar. Para solicitar una compresión de datos, la solicitud debe agregar el campo *Accept-Encoding* al encabezado de solicitud. Los valores aceptados son: gzip, Comprimir, desinflar, br, identidad, etc.

Diagrama de secuencias

Los diagramas de secuencia se utilizan para explicar una secuencia de intercambios o eventos.

El diagrama de secuencia de solicitud de API tiene tres secuencias separadas:

- **Crear sesión:** La *solicitud de inicio de sesión* se etiqueta como HTTPS. Ésta crea sesión con credenciales.
- **Obtener dispositivos:** Solicite una lista de dispositivos (URI) de la plataforma.
- **Crear dispositivo:** Empieza con una solicitud POST para crear un dispositivo.



WebHook

Los WebHooks, también se conocen como *reverse API*, son APIs que pueden devolver llamadas HTTP a aplicaciones que se han suscrito a ellos.

Con webhooks, las aplicaciones son más eficientes ya que no se requieren mecanismos de sondeo. El servidor que corre la API devuelve una llamada HTTP, o un *HTTP POST*, a una URI (especificada por la aplicación cuando cuando se suscribió) notificando cuando éste produce una actividad o evento en particular en los recursos.

Varias aplicaciones pueden suscribirse a un mismo servidor WebHook. Ellas, para recibir una notificación de un proveedor, debe cumplir ciertos requisitos:

- Debe ejecutarse en todo momento.
- Debe registrar un URI en el proveedor WebHook.
- Deben poder manejar las notificaciones entrantes del servidor WebHook.

Un ejemplos de servidores WebHook podría ser la plataforma Cisco DNA Center que permiten a las aplicaciones de terceros recibir datos de red cuando se producen eventos especificados

Autenticación de REST API

Algunas REST API que no requieren autenticación son de sólo lectura y no contienen información crítica o confidencial. Las otras requieren autenticación para que usuarios aleatorios no puedan acceder, crear, actualizar o eliminar información de forma incorrecta o maliciosa.

La **autenticación** es el proceso en el que el usuario demuestra su identidad, similar a cuando presentamos nuestro DNI para hacer algún trámite. Luego de la autenticación, según los permisos que tenga el usuario, el sistema nos dará **autorización** de acceso a los recursos.

Mecanismos de autenticación

Los tipos comunes de mecanismos de autenticación incluyen:

- **Autenticación básica:** Transmite credenciales como pares de nombre de usuario/contraseña separados con dos puntos (:) y codificados usando Base64.
- **Autenticación al portador:** Utiliza un token al portador, que es una cadena generada por un servidor de autenticación como un servicio de identidad (ID).
- **Clave API:** Es una cadena alfanumérica única generada por el servidor y asignada a un usuario. Los dos tipos de claves son *públicas* y *privadas*, éstas están emparejadas: A una clave pública le corresponde una clave privada.

Mecanismos de autorización

REST API usa **Open Authorization** (OAuth), que combina la autenticación con la autorización. Éste es un estándar abierto que permite flujos simples de autorización para sitios web o aplicaciones informáticas. Se trata de un protocolo que da la autorización segura de una API de modo estándar y simple para aplicaciones de escritorio, móviles y web.

OAuth permite al usuario proporcionar credenciales directamente al servidor de autorización [*Identity Provider* (IdP) o a *Identity Service* (ID)] quien le brindará un **token** de acceso para

compartir con la aplicación, habilitando a un usuario del sitio A (proveedor de servicio) compartir su información con el sitio B (consumidor) sin compartir toda su identidad. Para desarrolladores de *consumidores*, OAuth es un método de interactuar con datos protegidos y publicarlos; mientras que para desarrolladores de *proveedores* de servicio proporciona a los usuarios un acceso a sus datos al mismo tiempo que protege las credenciales de su cuenta.

OAuth fue desarrollado como una solución para mecanismos de autenticación inseguros. Hay dos versiones de **OAuth 1.0** y **OAuth 2.0**, donde OAuth 2.0 no es compatible con versiones anteriores.

Límite de velocidad de la REST API

Un límite de velocidad de API es una forma que un servicio web controle el número de solicitudes que un usuario o una aplicación puede realizar por unidad de tiempo definida. El límite de velocidad ayuda a evitar una sobrecarga del servidor por exceso de solicitudes, proporcionar un mejor servicio y tiempo de respuesta a todos los usuarios y protege en contra de un ataque de **Denial-of-Service** (DoS).

Existen 4 algoritmos de limitación de velocidad: Contador dinámico, Depósito de token, contador de ventana fija y contador de ventana deslizante.

Muchas APIs agregan detalles sobre el límite de velocidad en el encabezado de la respuesta. Las claves de límite de velocidad comúnmente utilizadas incluyen:

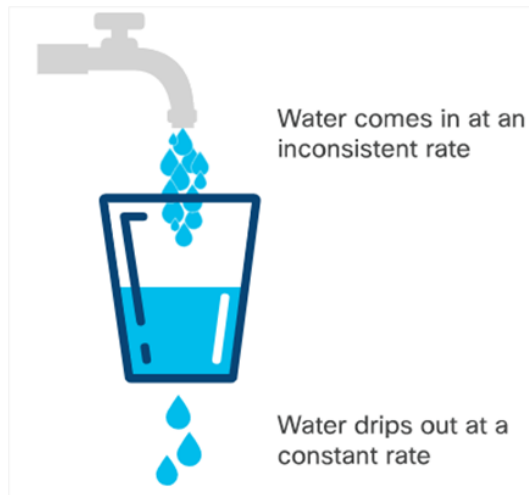
- **Límite de velocidad X:** El número máximo de solicitudes que se pueden realizar en una unidad de tiempo especificada.
- **Límite de tasa X restante:** El número de solicitudes pendientes que el solicitante puede realizar en la ventana de límite de tasa actual.
- **X-Rate Limit-Reset:** La hora en que se restablecerá la ventana de límite de velocidad.

Cuando se supera el límite de velocidad, el servidor rechaza automáticamente la solicitud y devuelve una respuesta HTTP al usuario. Ésta contiene el error "*límite de velocidad excedido*" y un código de estado HTTP significativo (429: *Demasiadas solicitudes* o 403: *Prohibido*)

Algoritmos de límite de velocidad

Contador dinámico:

Este algoritmo pone todas las solicitudes entrantes en una *cola de solicitudes* en el orden de llegada, pero el servidor procesa a una tasa fija. Si la cola de solicitudes está llena, las nuevas solicitudes son rechazadas.

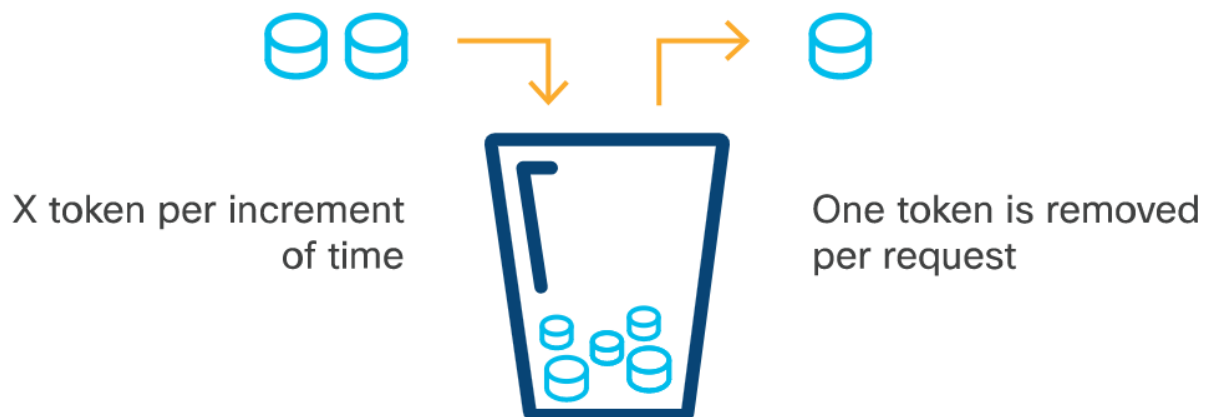


Un balde agujereado en su parte inferior es una analogía a éste algoritmo: Éste se desbordará si la velocidad promedio a la que recibe agua supera la velocidad (casi) constante a la que se vacía (o si más agua de la que es capaz de recibir es vertida de una sola vez).

Aplicando la analogía, el balde representa la capacidad máxima del servidor, el agujero la velocidad a la que se pueden despachar las peticiones, y el desbordamiento ocurre cuando el servidor es incapaz de despachar las peticiones.

Depósito de token

Este algoritmo proporciona a cada usuario un número definido de *tokens* que pueden usar dentro de un cierto incremento de tiempo. Cuando el cliente realiza una solicitud, el servidor comprueba el depósito para asegurarse de que contiene al menos un token. Si es así, elimina ese token y procesa la solicitud. Si no hay un token disponible, rechaza la solicitud. El cliente debe calcular el número de tokens que tiene actualmente para evitar solicitudes rechazadas.



Token bucket algorithm model

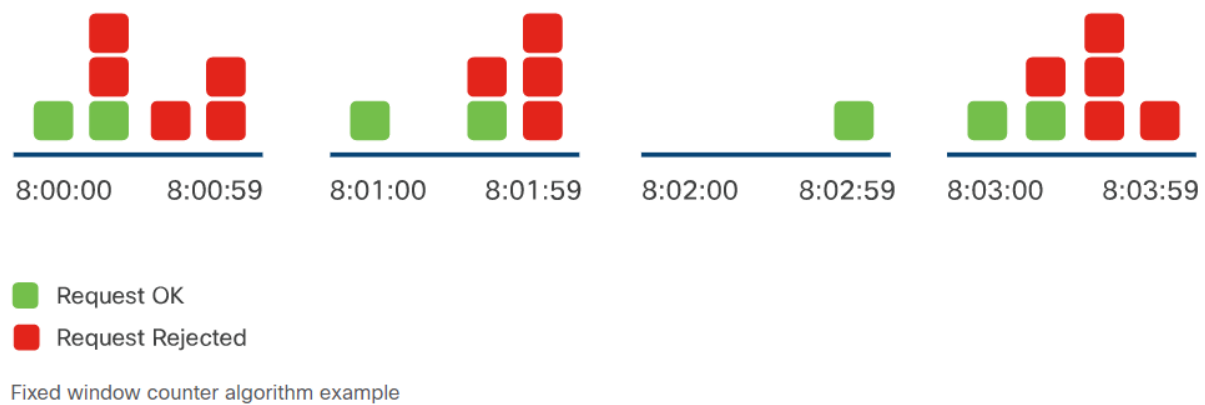
Contador de ventana fija

La *ventana fija* permite un número N de solicitudes de un usuarios en un período de tiempo determinado. Éste algoritmo le asigna un contador para representar cuántas solicitudes se han recibido durante ese período.

Al comienzo de cada período de tiempo el contador debe estar en cero, e ir incrementando su valor conforme se van recibiendo solicitudes y decrementando con cada solicitud procesada.

Si se cumple el límite de la ventana para ese período de tiempo, todas las solicitudes posteriores dentro de ese período de tiempo serán rechazadas.

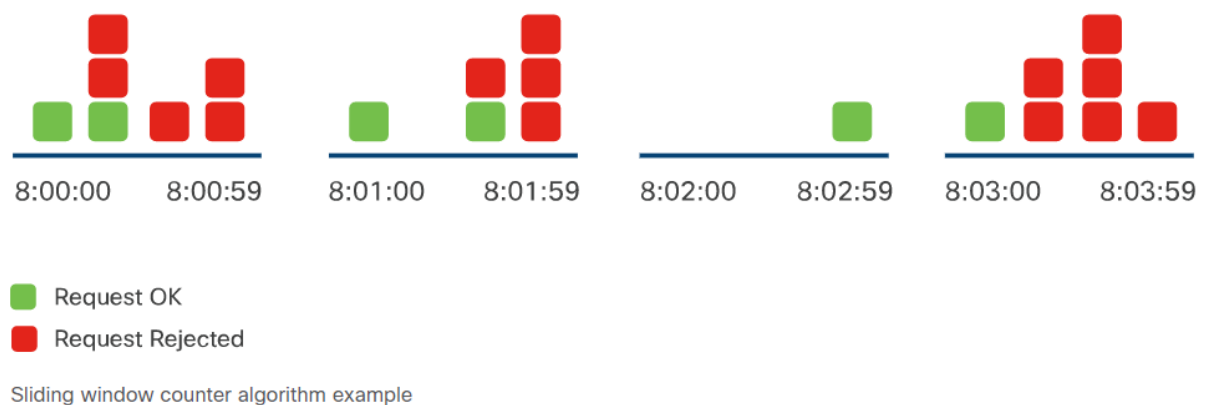
Replenish Rate: 2 Counters / Min



Contador de ventana deslizable

Este algoritmo resuelve los problemas de ráfagas de solicitudes con el algoritmo de ventana fija iniciando la ventana de tiempo cuando se realiza una solicitud. A diferencia de la ventana fija, la ventana de tiempo comienza solo cuando un usuario realmente realiza la primera solicitud. La marca de tiempo de la primera solicitud se guarda con un contador y el usuario puede realizar una solicitud más dentro de ese minuto. El cliente debe asegurarse de que el límite de solicitudes no esté excedido en el momento de realizarla.

Rate: 2 Requests / Min



Fuentes

<https://www.redhat.com/es/topics/api/what-are-application-programming-interfaces>
https://es.wikipedia.org/wiki/Simple_Object_Access_Protocol
<https://es.wikipedia.org/wiki/WebSocket>
https://es.wikipedia.org/wiki/Extensible_Markup_Language
<https://community.cisco.com/t5/developer-general-knowledge-base/xml-vs-json-vs-yaml/ta-p/4729758>
https://es.wikipedia.org/wiki/Espacio_de_nombres_XML
<https://es.wikipedia.org/wiki/JSON>
<https://es.wikipedia.org/wiki/YAML>
<https://www.tibco.com/es/reference-center/what-is-api-throttling>

Material del curso Cisco DevNet